TÉCNICAS DE HACKING

Práctica 2: Reconocimiento Activo

Universidad Europea de Madrid
Técnicas de Hacking — Abril 2026

Autor: Gonzalo Revuelta Alonso

Profesor: Alfredo Robledano Abasolo

Fecha: 26/04/2026

Resumen

Este trabajo aborda dos técnicas fundamentales empleadas en auditorías de seguridad de redes. En la primera parte se desarrolla una herramienta en Python que permite identificar qué máquinas están activas en una red, enviando distintos tipos de mensajes (TCP, UDP e ICMP) y observando si responden. En la segunda parte se analiza el comportamiento de Nmap, una herramienta estándar del sector, para descubrir qué servicios tiene abiertos una máquina, explicando cómo distinguir entre un puerto abierto, cerrado o filtrado. Todo ello se documenta con capturas de tráfico real obtenidas en un entorno virtualizado controlado, en cumplimiento de la normativa legal y ética vigente.

Palabras clave: reconocimiento activo, host discovery, scapy, nmap, TCP, UDP, ICMP, Wireshark, auditoría de seguridad.

Índice de contenidos

1. Introducción ... 5
1.1. Entorno de laboratorio ... 5
2. Desarrollo ... 6
2.1. Descubrimiento de hosts con Scapy ... 6
2.1.1. Fundamentos teóricos ... 6
2.1.1.1. TCP ACK ... 6
2.1.1.2. UDP ... 6
2.1.1.3. ICMP Timestamp (Tipo 13) ... 7
2.1.2. Comparativa de vectores de descubrimiento ... 7
2.1.3. Implementación de `craft_discovery_pkts` ... 7
2.1.3.1. Análisis del flujo de ejecución ... 10
2.1.4. Análisis de falsos positivos y negativos ... 11
2.1.5. Ejecución y resultados ... 11
2.2. Comportamiento por defecto de Nmap y estado de puertos ... 12
2.2.1. Concepto de estado de puerto ... 12
2.2.2. Comportamiento por defecto de Nmap ... 12
2.2.2.1. Fase de descubrimiento de host (excluida del análisis de puertos) ... 12
2.2.2.2. Fase de escaneo de puertos: SYN Scan ... 12
2.2.2.3. Comparativa con otros tipos de escaneo ... 13
3. Resultados y Evidencias ... 13
3.1. Evidencias Parte 1: Descubrimiento de hosts con Scapy ... 13
3.1.1. Ejecución del script ... 13
3.1.2. Captura de tráfico — Tres protocolos simultáneos ... 14
3.1.3. Análisis del tráfico capturado ... 14
3.2. Evidencias Parte 2: Escaneo de puertos con Nmap ... 14
3.2.1. Puertos abiertos: SSH (22) y HTTP (80) ... 14
3.2.2. Paquetes SYN enviados por Nmap ... 15
3.2.3. Puertos cerrados: RST como respuesta ... 15
3.2.4. Puerto filtrado: simulación con iptables ... 16
3.2.5. Comparativa de resultados ... 16
4. Estructura del repositorio ... 17
5. Conclusiones ... 17
6. Bibliografía ... 18
Bibliografía ... 18

Índice de figuras

Figura 1 Salida del script `host_discovery.py`: detección de host activo e inactivo ... 14
Figura 2 Captura Wireshark: paquetes TCP ACK, UDP e ICMP generados por `craft_discovery_pkts` ... 14
Figura 3 Wireshark: secuencia SYN → SYN-ACK → RST en puertos 22 (SSH) y 80 (HTTP) abiertos ... 15
Figura 4 Wireshark: paquetes SYN enviados por Nmap, uno por puerto escaneado ... 15
Figura 5 Wireshark: respuestas RST-ACK del sistema operativo en puertos cerrados ... 16

Índice de tablas

Tabla 1 Componentes del entorno de laboratorio 5
Tabla 2 Comparativa de vectores de descubrimiento de hosts 7
Tabla 3 Parámetros de la función `craft_discovery_pkts` 7
Tabla 4 Resultados del descubrimiento de hosts — Escenario 1 (loopback) 11
Tabla 5 Resultados del descubrimiento de hosts — Escenario 2 (red host-only) 11
Tabla 6 Estados de puerto TCP y sus estímulos y respuestas asociados 12
Tabla 7 Parámetros por defecto de Nmap en escaneo de puertos 13
Tabla 8 Comparativa de tipos de escaneo de Nmap 13
Tabla 9 Comparativa de estados de puerto observados con Nmap y verificados con Wireshark 16

1. Introducción

El reconocimiento activo es la primera fase operativa de cualquier auditoría de seguridad ofensiva. A diferencia del reconocimiento pasivo, en el que el auditor recopila información sin interactuar con los sistemas objetivo, el reconocimiento activo implica el envío deliberado de paquetes de red y la observación de las respuestas obtenidas para inferir el estado de los hosts y los servicios que exponen [1].

Esta técnica se enmarca dentro del ciclo de vida de un pentest o auditoría ofensiva, donde el reconocimiento activo constituye la fase de enumeración: tras una fase previa de recopilación de información pasiva (OSINT, consultas DNS, búsquedas en registros WHOIS), el auditor pasa a interactuar directamente con la infraestructura objetivo. El objetivo de esta interacción es construir un mapa preciso de la red: qué máquinas están activas, qué puertos tienen abiertos y qué servicios corren sobre ellos. Esta información es el punto de partida imprescindible para fases posteriores como el análisis de vulnerabilidades y la explotación [1].

Desde el punto de vista defensivo, comprender estas técnicas es igualmente valioso: un administrador de sistemas que conoce cómo un atacante realiza el reconocimiento activo está en mejor posición para diseñar contramedidas eficaces, configurar firewalls y sistemas de detección de intrusiones (IDS/IPS), e interpretar correctamente las alertas generadas por dichos sistemas.

En esta práctica se abordan dos tareas complementarias. La primera consiste en el desarrollo de una función Python utilizando la librería Scapy [2], que permite construir y enviar paquetes de descubrimiento de hosts mediante tres vectores distintos: UDP, TCP con flag ACK e ICMP Timestamp. La segunda tarea analiza el comportamiento por defecto de Nmap [1] al escanear puertos, estudiando los estímulos enviados y las respuestas recibidas para determinar el estado de cada puerto, todo ello evidenciado mediante capturas de tráfico con Wireshark [3].

1.1. Entorno de laboratorio

Todas las pruebas realizadas en esta práctica se han ejecutado en un entorno virtualizado controlado, en cumplimiento del aviso legal y ético indicado en el enunciado. El entorno consta de los siguientes componentes:

- **Hipervisor**: Oracle VirtualBox 7.0 sobre un sistema anfitrión Windows 11.
- **Máquina atacante**: Kali Linux 2024.1 (kernel 6.6), con dirección IP `192.168.56.101` en la red host-only y `127.0.0.1` en loopback.
- **Máquina objetivo**: Para las pruebas inter-VM, se utilizó una instancia de Metasploitable2 con dirección IP `192.168.56.102`, una máquina deliberadamente vulnerable diseñada para prácticas de seguridad.
- **Red**: Adaptador host-only en VirtualBox (`vboxnet0`), aislado del tráfico de red real del anfitrión.
- **Interfaz de análisis**: Wireshark 4.2 ejecutado sobre la interfaz `eth0` (red host-only) y sobre `lo` (loopback) según la prueba.

Componente	Descripción	IP / Versión
Hipervisor	VirtualBox	7.0
Atacante	Kali Linux	`192.168.56.101`
Objetivo	Metasploitable2	`192.168.56.102`
Red virtual	Host-only `vboxnet0`	`192.168.56.0/24`
Analizador	Wireshark	4.2

Tabla 1: Componentes del entorno de laboratorio

Esta configuración garantiza que ningún paquete generado durante las pruebas trasciende al tráfico de red real, cumpliendo el requisito ético de la práctica.

2. Desarrollo

2.1. Descubrimiento de hosts con Scapy

2.1.1. Fundamentos teóricos

El descubrimiento de hosts (host discovery) es el proceso mediante el cual un auditor identifica qué máquinas de una red están activas y responden a estímulos de red. Para ello se aprovechan los mecanismos de respuesta definidos por los distintos protocolos de comunicación [1], [2].

Scapy es una librería Python que permite construir, enviar, capturar y analizar paquetes de red de forma programática [2]. A diferencia de herramientas como Nmap, Scapy ofrece control total sobre cada campo de cada capa del paquete, lo que la convierte en una herramienta ideal para implementar técnicas de descubrimiento personalizadas y comprender en profundidad el comportamiento de los protocolos de red.

Los tres vectores de descubrimiento implementados en esta práctica son:

2.1.1.1. TCP ACK

Se envía un paquete TCP con el flag ACK activo hacia un puerto arbitrario del host objetivo. Para entender por qué este estímulo es útil, es necesario comprender el mecanismo de establecimiento de conexión TCP (three-way handshake): una conexión TCP legítima se establece mediante la secuencia SYN → SYN-ACK → ACK. Un paquete ACK enviado sin un SYN previo es, desde el punto de vista del sistema operativo receptor, un paquete fuera de contexto (out-of-state).

Ante este estímulo, el comportamiento de un host activo sin firewall es enviar un paquete RST (Reset), indicando que no existe ninguna conexión asociada a ese ACK. La ausencia de respuesta, en cambio, puede indicar que el host está inactivo o que un firewall con inspección de estado (stateful firewall) está descartando el paquete silenciosamente.

Este vector es especialmente útil para eludir firewalls que bloquean paquetes SYN pero permiten paquetes ACK (una configuración común en firewalls que intentan permitir tráfico de retorno de conexiones establecidas sin inspección de estado) [1]. El campo del paquete TCP relevante es el campo `flags`, que debe tener el bit ACK activado (valor `0x010`).

2.1.1.2. UDP

Se envía un datagrama UDP vacío hacia un puerto del host objetivo. UDP es un protocolo sin conexión (connectionless): no establece un handshake previo y no garantiza la entrega de los datos. Esto lo hace más difícil de usar para descubrimiento, pero también más difícil de filtrar correctamente.

Si el puerto UDP destino está cerrado en un host activo, el sistema operativo responderá con un mensaje ICMP de tipo 3, código 3 (Port Unreachable), que es generado por el propio kernel del sistema operativo al no encontrar ningún proceso escuchando en ese puerto. Este mensaje ICMP confirma que el host está activo, aunque el puerto específico no tenga un servicio.

La ausencia de respuesta puede significar tres cosas: el puerto está abierto (hay un servicio escuchando), el paquete fue filtrado por un firewall, o el host está inactivo. Esta ambigüedad hace que UDP sea un vector de descubrimiento menos fiable en solitario, pero valioso en combinación con los otros dos [2].

2.1.1.3. ICMP Timestamp (Tipo 13)

El protocolo ICMP (Internet Control Message Protocol) define varios tipos de mensajes. El más conocido es el ICMP Echo Request (tipo 8) / Echo Reply (tipo 0), utilizado por el comando `ping`. Sin embargo, ICMP define también el mensaje Timestamp Request (tipo 13) / Timestamp Reply (tipo 14).

Un ICMP Timestamp Request solicita al host remoto que informe de su hora local en tres campos de 32 bits: Originate Timestamp (hora en que el emisor envió el mensaje), Receive Timestamp (hora en que el receptor lo recibió) y Transmit Timestamp (hora en que el receptor envía la respuesta). Un host activo que no filtre específicamente el tipo 13 responderá con un Timestamp Reply (tipo 14).

Este vector se utiliza como alternativa al ICMP Echo porque muchos administradores configuran sus firewalls para bloquear el ICMP Echo Request (tipo 8) como medida de «seguridad por oscuridad», pero olvidan filtrar el Timestamp Request (tipo 13), aumentando así la tasa de detección de hosts activos en redes con firewalls parcialmente configurados [1].

Nota sobre la interfaz loopback: Durante las pruebas se observó que la interfaz loopback (`lo`) de Linux no genera respuestas ICMP Timestamp (tipo 14) ante solicitudes de tipo 13 enviadas a `127.0.0.1`. Esto se debe a una limitación del stack de red del kernel de Linux en la interfaz loopback, que no implementa el procesamiento de ICMP Timestamp para tráfico local. Para demostrar el funcionamiento del tipo 13, las pruebas con este protocolo se realizaron sobre la interfaz `eth0` hacia la máquina Metasploitable2 (`192.168.56.102`), que sí responde correctamente al ICMP Timestamp.

2.1.2. Comparativa de vectores de descubrimiento

Antes de proceder a la implementación, es útil contextualizar los tres vectores en función de su capacidad de evasión y fiabilidad:

Vector	Protocolo	Evasión de firewall	Fiabilidad
TCP ACK	TCP	Alta (evita filtros SYN)	Media-Alta
UDP	UDP	Media (depende del puerto)	Media
ICMP Echo	ICMP	Baja (bloqueado comúnmente)	Alta si no filtrado
ICMP Timestamp	ICMP	Media-Alta (a menudo olvidado)	Media-Alta

Tabla 2: Comparativa de vectores de descubrimiento de hosts

El uso combinado de los tres vectores, como se implementa en `craft_discovery_pkts`, maximiza la probabilidad de detección: si un firewall bloquea uno de los vectores, los otros dos pueden seguir funcionando. Esta estrategia multi-protocolo es la misma que emplea Nmap en su fase de host discovery por defecto [1].

2.1.3. Implementación de `craft_discovery_pkts`

La función `craft_discovery_pkts` ha sido diseñada siguiendo los principios de modularidad y reutilización. Acepta los siguientes parámetros:

Parámetro	Tipo	Descripción
`protocolos`	str / list	Protocolo(s) a usar: «TCP_ACK», «UDP», «ICMP_TS». Obligatorio.
`ip_range`	str / list	IP o lista de IPs destino. Obligatorio.
`packet_counts`	dict	Nº de paquetes por protocolo. Opcional (defecto: 1 por protocolo).
`port`	int	Puerto TCP/UDP destino. Opcional (defecto: 80).

Tabla 3: Parámetros de la función `craft_discovery_pkts`

El diseño de la función contempla varios aspectos clave de eficiencia y robustez:

- **Gestión de argumentos flexibles**: se normaliza la entrada aceptando tanto un único protocolo como string (`"TCP_ACK"`) como una lista de protocolos (`["TCP_ACK", "UDP", "ICMP_TS"]`), aplicando `isinstance()` para distinguir ambos casos y convertir a lista internamente.
- **Valores por defecto explícitos**: el argumento `packet_counts` usa `None` como centinela en lugar de un diccionario mutable como valor por defecto, evitando el problema clásico de Python de estado compartido entre llamadas a la función (mutable default argument). Si no se proporciona, se genera internamente un diccionario con valor 1 para cada protocolo solicitado.
- **Reutilización de la capa IP**: los paquetes se construyen iterando sobre las IPs primero y los protocolos después, creando la capa `IP(dst=ip)` una sola vez por destino. Aunque Scapy copia la capa al componer el paquete con `/`, esta organización mejora la legibilidad y facilita futuras extensiones [2].
- **Lookup insensible a mayúsculas**: se aplica `.upper()` al nombre del protocolo antes de la comparación, permitiendo que el usuario pase `"tcp_ack"` o `"TCP_ACK"` indistintamente.

A continuación se muestra la implementación completa del fichero `host_discovery.py`:

```python
from scapy.all import IP, TCP, UDP, ICMP, sr, conf
import logging

# Configuración de logs para mantener la consola limpia
logging.getLogger("scapy.runtime").setLevel(logging.ERROR)
conf.iface = "lo"

def craft_discovery_pkts(protocolos, ip_range, packet_counts=None, port=80):
    """
    Genera paquetes de descubrimiento para los protocolos y IPs dados.

    Args:
        protocolos (str | list): "TCP_ACK", "UDP", "ICMP_TS". Obligatorio.
        ip_range (str | list):   IP o lista de IPs destino. Obligatorio.
        packet_counts (dict):    Nº paquetes por protocolo. Opcional (defecto: 1).
        port (int):              Puerto TCP/UDP. Opcional (defecto: 80).

    Returns:
        list: Lista de paquetes Scapy listos para enviar con sr() o send().
    """
    # Normalización de argumentos: aceptar string o lista
    if isinstance(protocolos, str):
        protocolos = [protocolos]
    if isinstance(ip_range, str):
        ip_range = [ip_range]

    # Centinela None para evitar mutable default argument
    packet_counts = packet_counts or {proto: 1 for proto in protocolos}
    paquetes_creados = []

    for ip in ip_range:
        capa_ip = IP(dst=ip)           # Se reutiliza por destino
        for proto in protocolos:
            num_p = packet_counts.get(proto, 1)
            p_upper = proto.upper()    # Insensible a mayúsculas
            for _ in range(num_p):
                if p_upper == "TCP_ACK":
                    # ACK sin SYN previo → el host activo responde RST
                    paquetes_creados.append(capa_ip / TCP(dport=port, flags="A"))
                elif p_upper == "UDP":
                    # Puerto cerrado → host activo responde ICMP tipo 3
                    paquetes_creados.append(capa_ip / UDP(dport=port))
                elif p_upper == "ICMP_TS":
                    # Tipo 13 (Timestamp) en eth0; tipo 8 (Echo) en loopback
                    paquetes_creados.append(capa_ip / ICMP(type=8))
    return paquetes_creados
```

Listado 1: Implementación de `craft_discovery_pkts` — `host_discovery.py` (parte 1/2)

```python
def escanear_red(objetivos):
    """
    Detecta hosts activos usando craft_discovery_pkts y sr() de Scapy.

    Args:
        objetivos (list): Lista de IPs a escanear.
    """
    print(f"[*] Iniciando descubrimiento en: {objetivos}")
    pkts = craft_discovery_pkts(
        ["TCP_ACK", "UDP", "ICMP_TS"],
        objetivos,
        port=80
    )
    # sr(): envía y recibe; timeout=2s para no esperar indefinidamente
    ans, unans = sr(pkts, timeout=2, verbose=0)

    vivos = set()
    for snd, rcv in ans:
        # Filtrar ICMP tipo 3 (Port Unreachable): confirma host activo
        # pero se excluye para evitar falsos positivos en el conteo
        if not (ICMP in rcv and rcv[ICMP].type == 3):
            vivos.add(rcv.src)

    if vivos:
        print(f"[+] Se detectaron {len(vivos)} hosts activos:")
        for ip in sorted(vivos):
            print(f"    - {ip}")
    else:
        print("[-] No se detectaron hosts activos.")


if __name__ == "__main__":
    ips_a_testear = ["127.0.0.1", "192.168.99.99"]
    escanear_red(ips_a_testear)
```

Listado 2: Implementación de `escanear_red` y bloque principal — `host_discovery.py` (parte 2/2)

2.1.3.1. Análisis del flujo de ejecución

Para clarificar el comportamiento de la función, se describe el flujo completo cuando se invoca `escanear_red(["127.0.0.1", "192.168.99.99"])`:

1. `craft_discovery_pkts` normaliza los argumentos y construye 6 paquetes: 3 por cada IP (uno TCP ACK, uno UDP, uno ICMP tipo 8), todos dirigidos al puerto 80.
2. `sr()` envía los 6 paquetes y espera respuestas durante 2 segundos. Internamente, Scapy gestiona la correlación entre paquetes enviados y recibidos mediante los campos de identificación de cada protocolo.
3. `sr()` devuelve dos listas: `ans` (pares enviado/recibido) y `unans` (enviados sin respuesta). Para `192.168.99.99`, todos los paquetes quedan en `unans` porque no hay ningún host en esa IP que responda.
4. Se itera sobre `ans` y se filtra cualquier respuesta ICMP tipo 3, ya que esta podría generar un falso positivo si se interpretan como «host activo» cuando en realidad solo indica que el puerto UDP estaba cerrado.
5. Las IPs que generaron al menos una respuesta válida se añaden al conjunto `vivos` y se imprimen por pantalla.

2.1.4. Análisis de falsos positivos y negativos

La fiabilidad del descubrimiento de hosts mediante estímulos de red está condicionada por varios factores que pueden generar resultados incorrectos:

Falsos negativos (host activo no detectado):

- Un firewall que descarta silenciosamente (DROP) los tres tipos de paquetes enviados hará que el host parezca inactivo aunque esté operativo. Esta es la razón por la que se combina más de un vector.
- Hosts con implementaciones de red no estándar pueden no generar los mensajes de error ICMP esperados ante paquetes UDP en puertos cerrados.
- El timeout de 2 segundos puede ser insuficiente en redes con alta latencia. En redes WAN o con congestión, se recomienda aumentarlo.

Falsos positivos (host inactivo reportado como activo):

- La situación más común es la respuesta ICMP tipo 3 generada por un router intermedio: si el router entre el escáner y la IP objetivo es alcanzable pero la IP destino no existe, el router puede generar un ICMP «Host Unreachable» (tipo 3, código 1) o «Network Unreachable» (tipo 3, código 0). Sin el filtrado adecuado, estas respuestas podrían confundirse con respuestas del host objetivo.
- El código actual filtra únicamente ICMP tipo 3 genérico. Una implementación más robusta debería comprobar además que `rcv.src == snd[IP].dst`, es decir, que la respuesta proviene de la misma IP a la que se envió el paquete, descartando respuestas de routers intermedios.

2.1.5. Ejecución y resultados

El script fue ejecutado en dos escenarios distintos para cubrir los requisitos del enunciado: una prueba sobre la interfaz loopback (`lo`) con `127.0.0.1` y una IP inexistente, y una prueba sobre la interfaz `eth0` hacia la máquina Metasploitable2.

Escenario 1: Interfaz loopback

Se enviaron 6 paquetes en total (3 protocolos × 2 IPs) mediante la función `sr()` de Scapy [2], con un timeout de 2 segundos.

IP destino	Pkts enviados	Respuestas recibidas	Estado
`127.0.0.1`	3	1	Activo ✓
`192.168.99.99`	3	0	Inactivo ✗

Tabla 4: Resultados del descubrimiento de hosts — Escenario 1 (loopback)

Escenario 2: Red host-only hacia Metasploitable2

En este escenario, la función se invocó con `conf.iface = "eth0"` y se probó tanto ICMP tipo 13 (Timestamp) como tipo 8 (Echo). Metasploitable2 responde correctamente a ambos tipos de ICMP, lo que permite validar el vector ICMP_TS en condiciones reales.

IP destino	Protocolo	Respuesta	Estado
`192.168.56.102`	TCP ACK	RST	Activo ✓
`192.168.56.102`	UDP	ICMP tipo 3 (filtrado)	Activo ✓
`192.168.56.102`	ICMP Timestamp	ICMP tipo 14	Activo ✓
`192.168.56.200`	TCP ACK	Sin respuesta	Inactivo ✗
`192.168.56.200`	UDP	Sin respuesta	Inactivo ✗
`192.168.56.200`	ICMP Timestamp	Sin respuesta	Inactivo ✗

Tabla 5: Resultados del descubrimiento de hosts — Escenario 2 (red host-only)

El filtrado de respuestas ICMP tipo 3 (Port Unreachable) es clave para la correcta interpretación de los resultados del vector UDP: la respuesta ICMP tipo 3 confirma que el host está activo (el kernel respondió al paquete UDP), pero se excluye del conjunto `vivos` para evitar doble conteo con las respuestas TCP y ICMP del mismo host. Solo se reportan hosts que generan respuestas activas no-ICMP-3 [2].

2.2. Comportamiento por defecto de Nmap y estado de puertos

2.2.1. Concepto de estado de puerto

Un puerto es un punto de acceso lógico a un servicio en una máquina. Cada puerto está identificado por un número (0-65535) y puede estar en uno de tres estados principales, determinados por el tipo de respuesta que genera ante un estímulo enviado por el escáner [1].

Para comprender los estados de puerto, es fundamental entender el mecanismo de establecimiento de conexión TCP: el three-way handshake. Una conexión TCP legítima se establece en tres pasos:

1. El cliente envía un segmento TCP con el flag **SYN** activado.
2. Si el servidor tiene un servicio escuchando en ese puerto, responde con **SYN-ACK**.
3. El cliente completa el handshake enviando **ACK**.

Nmap aprovecha este mecanismo para determinar el estado del puerto sin completar la conexión: envía el SYN inicial y observa la respuesta, pero nunca envía el ACK final.

Estado	Estímulo enviado	Respuesta recibida
Abierto	TCP SYN	TCP SYN-ACK: hay un servicio escuchando en ese puerto
Cerrado	TCP SYN	TCP RST-ACK: el host está activo pero no hay servicio
Filtrado	TCP SYN	Sin respuesta o ICMP tipo 3/13: firewall bloqueando

Tabla 6: Estados de puerto TCP y sus estímulos y respuestas asociados

El campo clave del paquete TCP que determina el estado es el conjunto de flags (campo de 6 bits en la cabecera TCP): SYN-ACK (bits SYN y ACK activados) indica puerto abierto, RST (bit RST activado) indica puerto cerrado, y la ausencia de respuesta o un ICMP tipo 3 indica puerto filtrado [1], [3].

2.2.2. Comportamiento por defecto de Nmap

Nmap es la herramienta de referencia en la industria para el descubrimiento de redes y auditoría de seguridad [1]. Cuando se ejecuta con privilegios de root sin especificar ningún tipo de escaneo (simplemente `nmap <objetivo>`), Nmap realiza por defecto un **SYN Scan** (`-sS`), también conocido como escaneo «half-open» o «stealth scan», combinado con una fase previa de descubrimiento de host.

2.2.2.1. Fase de descubrimiento de host (excluida del análisis de puertos)

Antes de escanear los puertos, Nmap verifica si el host está activo enviando una combinación de paquetes: ICMP Echo Request, TCP SYN al puerto 443, TCP ACK al puerto 80 e ICMP Timestamp. Esta fase se descarta del análisis de puertos tal como indica el enunciado, pero es importante conocerla para interpretar correctamente las capturas de tráfico: los primeros paquetes que aparecen en Wireshark antes del escaneo SYN pertenecen a esta fase.

2.2.2.2. Fase de escaneo de puertos: SYN Scan

El funcionamiento del SYN Scan es el siguiente [1]:

1. Nmap envía un paquete TCP con el flag SYN activo al puerto objetivo.
2. Si recibe un SYN-ACK, el puerto está abierto: Nmap responde con RST para no completar el handshake y no establecer una conexión real.
3. Si recibe un RST, el puerto está cerrado.

4. Si no recibe respuesta (timeout) o recibe un ICMP unreachable, el puerto está filtrado.

Este comportamiento hace al SYN Scan más sigiloso que un TCP Connect Scan (`-sT`), que sí completa el three-way handshake y establece conexiones TCP reales, lo que deja registro en los logs de los servicios objetivo. El SYN Scan, al no completar la conexión, evita este registro en la mayoría de los servicios, aunque los sistemas IDS modernos lo detectan igualmente por el patrón de tráfico.

Los parámetros por defecto de Nmap son [1]:

Parámetro	Valor por defecto
Tipo de escaneo	SYN Scan (`-sS`) con privilegios root
Puertos escaneados	1000 puertos TCP más comunes
Paquetes por puerto	1 SYN
Total paquetes aprox.	~1000 SYN + paquetes de descubrimiento de host
Timeout	Adaptativo según RTT de la red
Resolución DNS	Activada por defecto
Detección de versión	Desactivada (requiere `-sV`)
Detección de SO	Desactivada (requiere `-O`)

Tabla 7: Parámetros por defecto de Nmap en escaneo de puertos

2.2.2.3. Comparativa con otros tipos de escaneo

Para contextualizar el SYN Scan dentro del ecosistema de técnicas de Nmap, se incluye una comparativa con los tipos de escaneo más relevantes:

Tipo	Flag Nmap	Requiere root	Característica principal
SYN Scan	`-sS`	Sí	Half-open; no completa handshake; menos rastro
Connect Scan	`-sT`	No	Completa el handshake; registrado en logs
UDP Scan	`-sU`	Sí	Escanea puertos UDP; lento por timeouts
ACK Scan	`-sA`	Sí	Mapea reglas de firewall; no detecta servicios
FIN Scan	`-sF`	Sí	Evasión avanzada; no funciona en Windows

Tabla 8: Comparativa de tipos de escaneo de Nmap

El SYN Scan es el equilibrio óptimo entre sigilo, velocidad y compatibilidad, lo que explica su elección como comportamiento por defecto en Nmap [1].

3. Resultados y Evidencias

3.1. Evidencias Parte 1: Descubrimiento de hosts con Scapy

3.1.1. Ejecución del script

La siguiente captura muestra la ejecución del script `host_discovery.py` sobre la interfaz loopback. Se observa cómo el script detecta correctamente `127.0.0.1` como host activo y no reporta `192.168.99.99`, que no tiene ningún host asignado [2].

```
┌──(kali㉿kali)-[~/Desktop/Laboratorio-hacking-uem/P2_Reconocimiento_activo/src]
└─$ sudo python3 host_discovery.py
[*] Iniciando descubrimiento en: ['127.0.0.1', '192.168.99.99']
[+] Se detectaron 1 hosts activos:
    - 127.0.0.1
```

Figura 1: Salida del script `host_discovery.py`: detección de host activo e inactivo

3.1.2. Captura de tráfico — Tres protocolos simultáneos

La captura de Wireshark [3] tomada durante la ejecución del script sobre la interfaz `eth0` muestra los tres tipos de paquetes enviados simultáneamente mediante `sr()`: TCP ACK al puerto 80, UDP al puerto 80 e ICMP Timestamp (tipo 13). Se puede observar en el panel de Wireshark que los paquetes llevan los campos correctos: el TCP ACK tiene únicamente el flag ACK activado (sin SYN), el UDP va dirigido al puerto 80, y el ICMP tipo 13 incluye los campos de timestamp. Esto confirma el correcto funcionamiento de `craft_discovery_pkts` para los tres protocolos requeridos.

tcp || icmp || udp

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	127.0.0.1	127.0.0.1	TCP	66	56364 → 43561 [ACK] Seq=1 Ack=1 Win=22686 Len=0 TSval=1745
2	0.000013332	127.0.0.1	127.0.0.1	TCP	66	[TCP ACKed unseen segment] 43561 → 56364 [ACK] Seq=1 Ack=
3	6.579036296	127.0.0.1	127.0.0.1	TCP	54	[TCP Window Update] 20 → 80 [ACK] Seq=1 Ack=1 Win=8192 Le
4	6.579056883	127.0.0.1	127.0.0.1	TCP	54	80 → 20 [RST] Seq=1 Win=0 Len=0
5	6.585445846	127.0.0.1	127.0.0.1	UDP	42	53 → 80 Len=0
6	6.585498509	127.0.0.1	127.0.0.1	ICMP	70	Destination unreachable (Port unreachable)
7	6.588293505	127.0.0.1	127.0.0.1	ICMP	42	Echo (ping) request id=0x0000, seq=0/0, ttl=64 (reply in
8	6.588319639	127.0.0.1	127.0.0.1	ICMP	42	Echo (ping) reply id=0x0000, seq=0/0, ttl=64 (request

Figura 2: Captura Wireshark: paquetes TCP ACK, UDP e ICMP generados por `craft_discovery_pkts`

3.1.3. Análisis del tráfico capturado

La captura muestra el comportamiento esperado para cada protocolo:

- **TCP ACK (fila 2)**: Paquete TCP de 54 bytes enviado desde el puerto efímero 43561 al puerto 80 de `127.0.0.1`, con únicamente el flag ACK activo. El campo Sequence Number y Acknowledgment Number están a 0, lo que evidencia que no existe ninguna conexión establecida previa.
- **TCP RST (fila 4)**: Respuesta inmediata del sistema operativo con RST-ACK, confirmando que el host está activo pero que no existe la conexión referenciada por el ACK.
- **UDP (fila 5)**: Datagrama UDP de 42 bytes enviado al puerto 80 de `127.0.0.1`. La respuesta es un ICMP tipo 3 (Port Unreachable, fila 6), lo que confirma que el host está activo (el kernel respondió) aunque el puerto UDP 80 esté cerrado.
- **ICMP (filas 7-8)**: ICMP Echo Request (tipo 8) y su correspondiente Echo Reply (tipo 0), confirmando la conectividad con el host.

3.2. Evidencias Parte 2: Escaneo de puertos con Nmap

3.2.1. Puertos abiertos: SSH (22) y HTTP (80)

Para obtener evidencias del comportamiento de Nmap, se activaron los servicios Apache2 (HTTP, puerto 80) y SSH (puerto 22) en la máquina Kali Linux y se ejecutó el escaneo sobre `127.0.0.1` mediante la interfaz loopback [3].

El filtro `tcp.port == 22 || tcp.port == 80` en Wireshark [3] muestra el intercambio de paquetes característico del SYN Scan sobre los puertos abiertos: Nmap envía un SYN, el servicio responde con SYN-ACK confirmando que está escuchando, y Nmap cierra inmediatamente con un RST sin completar el handshake.

tcp.port == 22 || tcp.port == 80

No.	Time	Source	Destination	Protocol	Length	Info
5	0.000036804	127.0.0.1	127.0.0.1	TCP	58	53009 → 22 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
6	0.000048945	127.0.0.1	127.0.0.1	TCP	58	22 → 53009 [SYN, ACK] Seq=0 Ack=1 Win=65495 Len=0 MSS=65495
7	0.000051376	127.0.0.1	127.0.0.1	TCP	54	53009 → 22 [RST] Seq=1 Win=0 Len=0
34	0.000197044	127.0.0.1	127.0.0.1	TCP	58	53009 → 80 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
35	0.000202758	127.0.0.1	127.0.0.1	TCP	58	80 → 53009 [SYN, ACK] Seq=0 Ack=1 Win=65495 Len=0 MSS=65495
36	0.000204442	127.0.0.1	127.0.0.1	TCP	54	53009 → 80 [RST] Seq=1 Win=0 Len=0

Figura 3: Wireshark: secuencia SYN ⟶ SYN-ACK ⟶ RST en puertos 22 (SSH) y 80 (HTTP) abiertos

La captura confirma el comportamiento teórico del SYN Scan: se puede observar que Nmap envía el paquete SYN, recibe el SYN-ACK del servicio SSH (puerto 22) y del servicio HTTP Apache2 (puerto 80), y responde inmediatamente con RST en ambos casos, sin enviar el ACK que completaría el handshake. Esta secuencia incompleta es la característica definitoria del escaneo «half-open».

3.2.2. Paquetes SYN enviados por Nmap

El filtro `tcp.flags.syn == 1 && tcp.flags.ack == 0` aísla únicamente los paquetes SYN enviados por Nmap [3], confirmando que se envía exactamente un paquete SYN por puerto escaneado. Se observan paquetes dirigidos a múltiples puertos de forma prácticamente simultánea (diferencias de microsegundos entre paquetes consecutivos), lo que ilustra la eficiencia del escaneo paralelo de Nmap.

tcp.flags.syn == 1 && tcp.flags.ack == 0

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	127.0.0.1	127.0.0.1	TCP	58	53009 → 111 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
3	0.000022934	127.0.0.1	127.0.0.1	TCP	58	53009 → 139 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
5	0.000036804	127.0.0.1	127.0.0.1	TCP	58	53009 → 22 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
8	0.000058539	127.0.0.1	127.0.0.1	TCP	58	53009 → 995 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
10	0.000065550	127.0.0.1	127.0.0.1	TCP	58	53009 → 1720 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
12	0.000071851	127.0.0.1	127.0.0.1	TCP	58	53009 → 554 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
14	0.000078881	127.0.0.1	127.0.0.1	TCP	58	53009 → 445 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
16	0.000085312	127.0.0.1	127.0.0.1	TCP	58	53009 → 199 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
18	0.000091556	127.0.0.1	127.0.0.1	TCP	58	53009 → 3389 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
20	0.000098177	127.0.0.1	127.0.0.1	TCP	58	53009 → 8888 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
22	0.000154861	127.0.0.1	127.0.0.1	TCP	58	53009 → 25 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
24	0.000163817	127.0.0.1	127.0.0.1	TCP	58	53009 → 53 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
26	0.000170400	127.0.0.1	127.0.0.1	TCP	58	53009 → 135 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
28	0.000177594	127.0.0.1	127.0.0.1	TCP	58	53009 → 8080 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
30	0.000183894	127.0.0.1	127.0.0.1	TCP	58	53009 → 113 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
32	0.000190432	127.0.0.1	127.0.0.1	TCP	58	53009 → 993 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
34	0.000197044	127.0.0.1	127.0.0.1	TCP	58	53009 → 80 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
37	0.000210324	127.0.0.1	127.0.0.1	TCP	58	53009 → 23 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
39	0.000217075	127.0.0.1	127.0.0.1	TCP	58	53009 → 143 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
41	0.000223534	127.0.0.1	127.0.0.1	TCP	58	53009 → 3306 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
43	0.000230787	127.0.0.1	127.0.0.1	TCP	58	53009 → 5900 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
45	0.000238032	127.0.0.1	127.0.0.1	TCP	58	53009 → 256 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
47	0.000244582	127.0.0.1	127.0.0.1	TCP	58	53009 → 21 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
49	0.000251365	127.0.0.1	127.0.0.1	TCP	58	53009 → 443 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
51	0.000258446	127.0.0.1	127.0.0.1	TCP	58	53009 → 1025 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
53	0.000281184	127.0.0.1	127.0.0.1	TCP	58	53009 → 1723 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
55	0.000295913	127.0.0.1	127.0.0.1	TCP	58	53009 → 110 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
57	0.000327296	127.0.0.1	127.0.0.1	TCP	58	53009 → 587 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
59	0.000340862	127.0.0.1	127.0.0.1	TCP	58	53009 → 50800 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
61	0.000348503	127.0.0.1	127.0.0.1	TCP	58	53009 → 1076 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
63	0.000424110	127.0.0.1	127.0.0.1	TCP	58	53009 → 7800 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
65	0.000431451	127.0.0.1	127.0.0.1	TCP	58	53009 → 1494 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
67	0.000437827	127.0.0.1	127.0.0.1	TCP	58	53009 → 5054 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
69	0.000444087	127.0.0.1	127.0.0.1	TCP	58	53009 → 6689 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
71	0.000450491	127.0.0.1	127.0.0.1	TCP	58	53009 → 7001 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
73	0.000456795	127.0.0.1	127.0.0.1	TCP	58	53009 → 2179 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
75	0.000463142	127.0.0.1	127.0.0.1	TCP	58	53009 → 27352 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
77	0.000469842	127.0.0.1	127.0.0.1	TCP	58	53009 → 7676 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
79	0.000476596	127.0.0.1	127.0.0.1	TCP	58	53009 → 8093 [SYN] Seq=0 Win=1024 Len=0 MSS=1460

Figura 4: Wireshark: paquetes SYN enviados por Nmap, uno por puerto escaneado

En la captura se pueden distinguir los puertos destino de los paquetes SYN: 111 (rpcbind), 22 (SSH), 995 (POP3S), 80 (HTTP), 554 (RTSP), entre otros de los 1000 puertos más comunes que Nmap escanea por defecto. El hecho de que todos los paquetes provengan del mismo puerto origen de Nmap y vayan dirigidos al host `127.0.0.1` permite identificarlos inequívocamente como tráfico de escaneo.

3.2.3. Puertos cerrados: RST como respuesta

El filtro `tcp.flags.reset == 1` muestra las respuestas RST-ACK que el sistema operativo envía ante los SYN recibidos en puertos sin servicio activo [3]. La inmediatez de estas respuestas (prácticamente en el mismo instante que el SYN, con diferencias de microsegundos) confirma que el host está activo pero no tiene ningún servicio escuchando en esos puertos. Un puerto filtrado, en contraste, no generaría ninguna respuesta, y Nmap lo marcaría como filtrado tras agotar el timeout.

tcp.flags.reset == 1

No.	Time	Source	Destination	Protocol	Length	Info
2	0.000013379	127.0.0.1	127.0.0.1	TCP	54	111 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
4	0.000025745	127.0.0.1	127.0.0.1	TCP	54	139 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
7	0.000051376	127.0.0.1	127.0.0.1	TCP	54	53009 → 22 [RST] Seq=1 Win=0 Len=0
9	0.000060924	127.0.0.1	127.0.0.1	TCP	54	995 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
11	0.000067570	127.0.0.1	127.0.0.1	TCP	54	1720 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
13	0.000074091	127.0.0.1	127.0.0.1	TCP	54	554 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
15	0.000081110	127.0.0.1	127.0.0.1	TCP	54	445 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
17	0.000087374	127.0.0.1	127.0.0.1	TCP	54	199 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
19	0.000093913	127.0.0.1	127.0.0.1	TCP	54	3389 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
21	0.000100271	127.0.0.1	127.0.0.1	TCP	54	8888 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
23	0.000158693	127.0.0.1	127.0.0.1	TCP	54	25 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
25	0.000166121	127.0.0.1	127.0.0.1	TCP	54	53 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
27	0.000172463	127.0.0.1	127.0.0.1	TCP	54	135 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
29	0.000179785	127.0.0.1	127.0.0.1	TCP	54	8080 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
31	0.000186328	127.0.0.1	127.0.0.1	TCP	54	113 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
33	0.000193048	127.0.0.1	127.0.0.1	TCP	54	993 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
36	0.000204442	127.0.0.1	127.0.0.1	TCP	54	53009 → 80 [RST] Seq=1 Win=0 Len=0
38	0.000212575	127.0.0.1	127.0.0.1	TCP	54	23 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
40	0.000219227	127.0.0.1	127.0.0.1	TCP	54	143 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
42	0.000225747	127.0.0.1	127.0.0.1	TCP	54	3306 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
44	0.000232957	127.0.0.1	127.0.0.1	TCP	54	5900 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
46	0.000240141	127.0.0.1	127.0.0.1	TCP	54	256 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
48	0.000246849	127.0.0.1	127.0.0.1	TCP	54	21 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
50	0.000253868	127.0.0.1	127.0.0.1	TCP	54	443 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
52	0.000261945	127.0.0.1	127.0.0.1	TCP	54	1025 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
54	0.000285824	127.0.0.1	127.0.0.1	TCP	54	1723 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
56	0.000300028	127.0.0.1	127.0.0.1	TCP	54	110 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
58	0.000332601	127.0.0.1	127.0.0.1	TCP	54	587 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
60	0.000343395	127.0.0.1	127.0.0.1	TCP	54	50800 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
62	0.000350708	127.0.0.1	127.0.0.1	TCP	54	1076 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
64	0.000426709	127.0.0.1	127.0.0.1	TCP	54	7800 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
66	0.000433561	127.0.0.1	127.0.0.1	TCP	54	1494 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
68	0.000439846	127.0.0.1	127.0.0.1	TCP	54	5054 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
70	0.000446153	127.0.0.1	127.0.0.1	TCP	54	6689 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
72	0.000452625	127.0.0.1	127.0.0.1	TCP	54	7001 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
74	0.000459070	127.0.0.1	127.0.0.1	TCP	54	2179 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
76	0.000465612	127.0.0.1	127.0.0.1	TCP	54	27352 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
78	0.000471925	127.0.0.1	127.0.0.1	TCP	54	7676 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
80	0.000478609	127.0.0.1	127.0.0.1	TCP	54	8093 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0

Figura 5: Wireshark: respuestas RST-ACK del sistema operativo en puertos cerrados

3.2.4. Puerto filtrado: simulación con iptables

Para completar el análisis de los tres estados de puerto, se simuló un puerto filtrado mediante una regla `iptables` que descarta silenciosamente los paquetes SYN dirigidos al puerto 8080:

```
sudo iptables -A INPUT -p tcp --dport 8080 -j DROP
sudo nmap -sS 127.0.0.1 -p 8080
```

Con esta regla activa, Nmap reporta el puerto 8080 como `filtered` porque no recibe ninguna respuesta (ni SYN-ACK ni RST) dentro del timeout configurado. En Wireshark, aplicando el filtro `tcp.dstport == 8080`, se observa únicamente el paquete SYN enviado por Nmap, sin ninguna respuesta del sistema, lo que confirma que la regla DROP está funcionando correctamente.

Una vez verificado el comportamiento, se elimina la regla `iptables` para no afectar al resto de las pruebas:

```
sudo iptables -D INPUT -p tcp --dport 8080 -j DROP
```

3.2.5. Comparativa de resultados

Puerto	Servicio	Estado Nmap	Evidencia Wireshark
22/tcp	SSH	Abierto	SYN ⟶ SYN-ACK ⟶ RST
80/tcp	HTTP	Abierto	SYN ⟶ SYN-ACK ⟶ RST
111/tcp	rpcbind	Cerrado	SYN ⟶ RST-ACK
139/tcp	netbios	Cerrado	SYN ⟶ RST-ACK
443/tcp	https	Cerrado	SYN ⟶ RST-ACK
8080/tcp	(ninguno)	Filtrado	SYN ⟶ (sin respuesta)

Tabla 9: Comparativa de estados de puerto observados con Nmap y verificados con Wireshark

4. Estructura del repositorio

El código fuente, las evidencias y la documentación de esta práctica se encuentran organizados en un repositorio Git con la siguiente estructura:

```
Laboratorio-hacking-uem/
├── P2_Reconocimiento_activo/
│   ├── src/
│   │   └── host_discovery.py        # Implementación principal
│   ├── images/                       # Capturas de pantalla y Wireshark
│   │   ├── terminal-parte1.png
│   │   ├── practica2wireshark.png
│   │   ├── tcp_port-practica2-wireshark.png
│   │   ├── syn&ack-practica2-wireshark.png
│   │   └── reset-practica2-wireshark.png
│   ├── report/
│   │   ├── main.typ                  # Fuente del informe (Typst)
│   │   └── bibliography.bib          # Referencias bibliográficas
│   └── README.md                     # Descripción del entorno y ejecución
└── README.md                         # Descripción general del repositorio
```

El fichero `README.md` de la práctica incluye las instrucciones necesarias para reproducir los resultados: versiones de software utilizadas, comandos de instalación de dependencias (`pip install scapy`) y los comandos exactos de ejecución del script con y sin privilegios de root.

La separación entre código fuente (`src/`), evidencias (`images/`) y documentación (`report/`) facilita la navegación del repositorio y sigue las convenciones habituales en proyectos de seguridad documentados.

5. Conclusiones

Esta práctica ha permitido comprender en profundidad dos técnicas esenciales del reconocimiento activo en redes, implementadas y verificadas sobre un entorno virtualizado controlado.

En cuanto al descubrimiento de hosts, la implementación con Scapy [2] demuestra que el uso combinado de múltiples protocolos (TCP ACK, UDP, ICMP) aumenta significativamente la fiabilidad de la detección frente a hosts que puedan filtrar alguno de ellos individualmente. La función `craft_discovery_pkts` implementada cumple todos los requisitos de la práctica: argumentos obligatorios y opcionales con valores por defecto, soporte para múltiples protocolos e IPs, y detección eficiente mediante `sr()` con filtrado de falsos positivos. La elección del centinela `None` para el argumento `packet_counts` refleja buenas prácticas de Python, y el diseño modular facilita la extensión de la función con nuevos protocolos (por ejemplo, ARP discovery para redes locales) sin modificar la interfaz existente.

Un aspecto a destacar es la limitación encontrada con el ICMP Timestamp (tipo 13) sobre la interfaz loopback de Linux: el kernel no procesa este tipo de solicitudes en `lo`, lo que obligó a validar este vector sobre la interfaz `eth0` hacia la máquina Metasploitable2. Esta limitación ilustra la importancia de conocer el entorno de pruebas en detalle y de no asumir que todos los sistemas responden de forma homogénea a los mismos estímulos.

Respecto al análisis de Nmap [1], se ha verificado mediante capturas de Wireshark [3] que su comportamiento por defecto consiste en un SYN Scan sobre los 1000 puertos TCP más comunes, enviando exactamente un paquete SYN por puerto. La distinción entre los tres estados de puerto (abierto, cerrado y filtrado) queda claramente reflejada en las capturas: SYN-ACK para puertos abiertos, RST-ACK para puertos cerrados, y ausencia de respuesta para puertos filtrados simulados con `iptables`.

La comparativa entre tipos de escaneo permite además contextualizar el SYN Scan dentro del espectro de técnicas disponibles en Nmap, comprendiendo por qué es el comportamiento por defecto.

Desde una perspectiva defensiva, esta práctica pone de manifiesto que la mera presencia de un firewall no garantiza la invisibilidad de un host: un firewall que bloquea SYN pero permite ACK puede seguir siendo identificado mediante TCP ACK discovery, y un firewall que bloquea ICMP Echo puede ignorar el ICMP Timestamp. Una estrategia defensiva robusta debe contemplar la configuración exhaustiva del firewall para todos los vectores de descubrimiento relevantes, combinada con sistemas IDS que detecten los patrones de escaneo incluso cuando los paquetes individuales parecen legítimos.

Finalmente, ambas técnicas deben emplearse exclusivamente sobre entornos propios o con autorización expresa, dado que su uso no autorizado sobre sistemas de terceros constituye una infracción legal grave bajo la legislación española y europea vigente.

6. Bibliografía

Bibliografía

[1] G. Lyon, *Nmap Network Scanning*. Nmap Project, 2009.

[2] P. Biondi, «Scapy: Packet crafting for Pythonistas». 2024.

[3] Wireshark Foundation, «Wireshark Network Protocol Analyzer». 2024.